Université Grenoble Alpes
L3 Informatique

Année 2026-2027

# Architectures logicielles et matérielles
# Semaine 4 TP 1

*Le but de ces deux séances de TP sur carte STM32F4 sera de réaliser une* ***utilisation comparée du polling (attente active) et d'interruptions*** *lors de l'acquisition de données à partir d'un périphérique.*
*Il s'agit du RNG (Random Number Generator).*

## Informations préliminaires.

Le **Random Number Generator** (RNG) a pour fonction de générer des nombres *aléatoires*. Il s'agit d'un périphérique interne au microcontrôleur, il ne sera donc pas question ici de passer par les ports GPIO.
Quel est l'intérêt d'utiliser un tel générateur de nombres aléatoires plutôt qu'une fonction comme la fonction "rand" du C ? Les entiers générés par une fonction comme "rand" sont des entiers *pseudo-aléatoires.* Les générateurs pseudo-aléatoires calculent les valeurs successives d'une suite de récurrence de la forme $v_{i+1} = f(v_i)$ à partir d'un germe (une graine) $v_0$. La séquence générée semble aléatoire, mais elle est prédictible (et le même germe donnera toujours la même séquence). Pour rendre les valeurs moins prévisibles, on pourra calculer cette série à partir d'un germe différent à chaque exécution de l'application. Si cela est suffisant pour des applications comme une recherche aléatoire dans un tableau ou des jeux, on ne peut toutefois se contenter d'une telle solution pour générer une clef de sécurité par exemple (cryptographie). En effet, si un attaquant peut deviner le premier terme, alors il pourra deviner tous les autres. On peut obtenir de meilleurs résultats en exploitant le bruit thermique ou électromagnétique des circuits électroniques, selon diverses techniques. C'est ce que réalise le RNG ici, qui est capable de délivrer des *nombres aléatoires 32 bits* à une fréquence de l'ordre du Mhz.

**L'objectif** de ces deux TP est de comprendre comment ce composant RNG peut être utilisé (i.e., des nombres aléatoires peuvent être récupérés), via une solution à base de **polling** ou bien de **gestion d'interruptions**.

Dans ce TP 1 nous allons réaliser une **solution à base de *polling*** pour attendre la disponibilité d'une nouvelle valeur produite par le RNG (cf slide 14 du cours).

La documentation du RNG nous indique qu'il faut 40 cycles de l'horloge RNG_CLK avant la disponibilité du prochain nombre aléatoire. Mais évidemment, nous n'allons pas compter les cycles de cette horloge pour savoir à quels moments nous pouvons récupérer un nouveau nombre !
Dans la version de ce TP1, nous allons réaliser une **attente active sur le registre de status** du RNG *pour savoir si un nouveau nombre aléatoire est disponible*.
(et dans le TP 2, nous réaliserons une solution basée sur la gestion d'interruptions émises par le composant RNG)

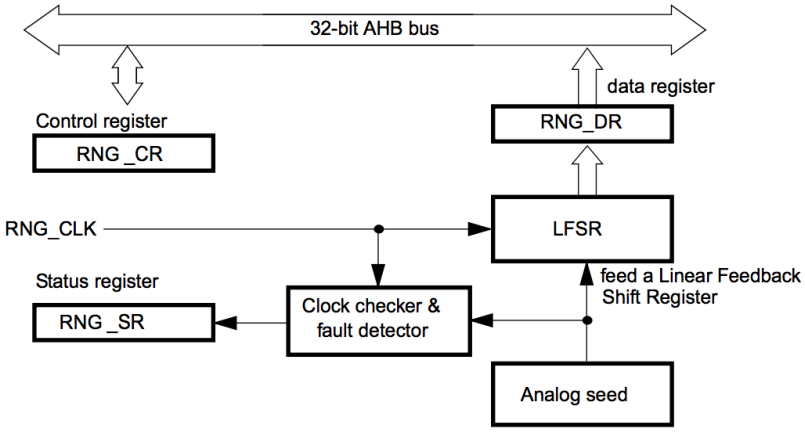
---

## Questions à traiter et remettre impérativement avant de commencer le TP.

**Question 1.**
Ce composant RNG présente les caractéristiques classiques d'un *périphérique avec coupleur d'entrées/sorties*, comme précédemment étudié en cours.

Rappeler le **rôle** des registres STATUS, CTRL et DATA d'un tel coupleur d'E/S.

Le document joint, *doc-rng.pdf*, est un extrait du manuel de référence ST. Il contient la section dédiée au RNG, qui illustre le block diagram comme ceci :



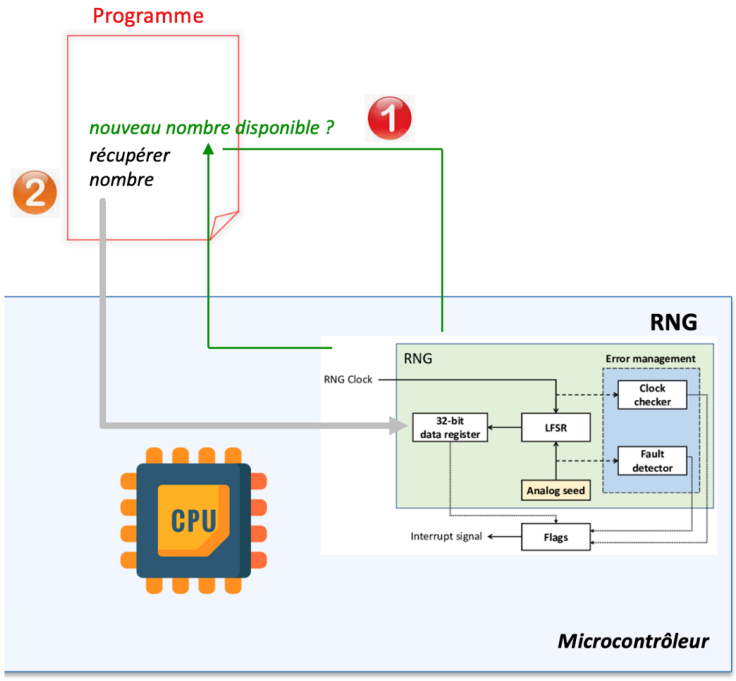

Reconnaître les 3 précédents **registres** sur cette figure, donner leurs noms.

On utilisera des macros permettant d'accéder aux contenus 32 bits de ces registres, qui sont données en Annexe. Préciser les noms et rôles de ces 3 **macros**.

**Question 2.**
Il faut avant tout être sûrs de maîtriser les principes nécessaires aux réalisations prévues dans ce TP1.

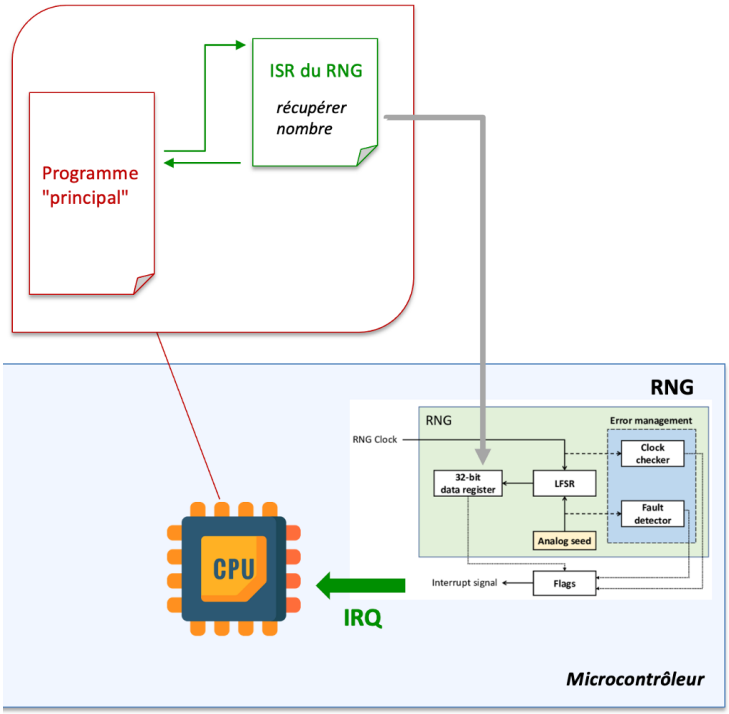
• Rappeler le **principe** du *polling*, le **registre** du périphérique qu'il faut consulter, ainsi que le **pseudo-code** (très simplifié) à utiliser dans ce cas.

• Expliquer soigneusement la vue schématique du traitement nécessaire expliquée en cours :
**où** (dans quelle fonction) devra se trouver la boucle de polling dans votre application ?
**que** sert-elle à attendre, et **où** (matériellement) devra-t-elle inspecter pour ce faire ?
**que** devra faire votre application à la **sortie** de cette attente active ?




**Question 3.**
Dans le TP 2, nous réaliserons une solution basée sur la gestion d'interruptions émises par le composant RNG. Expliquer soigneusement la vue schématique du traitement nécessaire expliquée en cours (cf slide 39), en comparant à la précédente :
**y aura-t-il** une boucle de polling dans votre application pour cette solution ?
**comment** et **quand** l'application sera-t-elle "prévenue" de la disponibilité d'un nouveau nombre ?
**en quoi consistera** le traitement alors nécessaire et **où** doit-il se trouver dans votre

application (et pourquoi) ?




---

Pour la suite, récupérer le fichier semaine4.tar dans le moodle et placez-le dans le répertoire de votre choix. Désarchiver le contenu.
Il contient comme précédemment un fichier *bp_leds.c*, et deux squelettes de programme pour les deux variantes que nous réaliserons dans les deux TP successifs, *firmware_polling.c* et *firmware_interrupt.c*
**Nous travaillerons ici avec le fichier *firmware_polling.c* de l'archive, à renommer en *firmware.c***
Note. Le fichier *bp_leds.c* sera complété à la question 5, ne pas s'en préoccuper pour l'instant.

**Question 1.**
La documentation du RNG nous indique qu'il faut avant tout *activer* la génération de nombres aléatoires en positionnant à 1 le bit RNGEN du registre RNG_CR, mais *ne pas activer* l'interruption du RNG (associée au bit IE du registre RNG_CR).

Faire le **lien** entre la représentation ci-dessous de ce registre **RNG_CR** et les constantes **`RNG_CR_RNGEN`** et **`RNG_CR_IE`** données en Annexe, que vous devrez utiliser dans la suite. Quelles valeurs représentent exactement ces 2 constantes?

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | | | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| Reserved | | | | | | | | | | | | IE | RNGEN | Reserved | |
| | | | | | | | | | | | | rw | rw | | |

Ecrire la **fonction**

```
void init_rng_polling();
```

pour mettre en œuvre les initialisations nécessaires (on n'utilisera aucune fonction de la bibliothèque libopencm3 pour écrire cette fonction).

**Question 2.**
La documentation précise que la disponibilité d'un nouveau nombre est repérée par la valeur 1 dans le bit DRDY du registre RNG_SR (il est important de noter que ce bit DRDY est automatiquement remis à 0 à la lecture de la donnée).
Mais attention, la documentation nous indique aussi que les bits SECS et CECS de ce registre

RNG_SR sont à 1 s'il y a eu respectivement une *seed error* (problème avec la graine pour la génération) ou une *clock error* (problème provenant de l'horloge).

Faire le **lien** entre la représentation ci-dessous de ce registre **RNG_SR** et les constantes **`RNG_SR_SECS`**, **`RNG_SR_CECS`** et **`RNG_SR_DRDY`** données en Annexe. Quelles valeurs représentent exactement ces 3 constantes ?

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | | | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| Reserved | | | | | | | | | SEIS | CEIS | Reserved | | SECS | CECS | DRDY |
| | | | | | | | | | rc_w0 | rc_w0 | | | r | r | r |

Ecrire les 3 **fonctions** suivantes (ici aussi, on n'utilisera aucune fonction de la libopencm3) :

```
int check_seed_error();     // renvoie 1 si seed error, 0 sinon
int check_clock_error();    // renvoie 1 si clock error, 0 sinon
int check_data_ready();     // renvoie 1 si donnée disponible, 0 sinon
```

**Question 3.**

Il reste maintenant à écrire (sans utilisation de fonctions de la libopencm3) la **fonction**

```
void wait_for_new_rnd();
```

dont le rôle est de réaliser une attente active jusqu'à ce qu'un nouveau nombre aléatoire soit disponible.

Attention, pendant cette attente active, si une erreur est détectée, il faut remettre à 1 le bit RNGEN du registre RNG_CR (pour réinitialiser et redémarrer le RNG).

**Question 4.**

Compléter le ***main*** fourni afin qu'à chaque disponibilité d'un nouveau nombre aléatoire, cette nouvelle valeur soit placée dans la variable globale **`rnd`**.

Nous pouvons ici commencer à **tester dans gdb**, afin de constater les mises à jour des valeurs de **`rnd`** :
Cross-compiler et charger l'exécutable.
Placer un breakpoint vous permettant d'observer la valeur de **`rnd`** après chaque mise à jour, continuer l'exécution jusqu'à ce breakpoint et faire afficher la valeur de **`rnd`**.
Reproduire la même expérience quelques fois : observer des valeurs successives de **`rnd`**, vous devez constater des changements de valeurs.
(attention, si vous n'observez pas de changements de valeurs, inutile de continuer car votre RNG n'est pas opérationnel, il faut d'abord corriger vos fonctions ci-dessus)

Pour pouvoir tester (visuellement) notre code, nous allons maintenant faire en sorte que les nombres aléatoires successivement produits nous servent à modifier aléatoirement le choix des **leds** allumées.

Mais attention : la génération de nouveaux nombres est tellement rapide que vous ne pourrez pas percevoir les changements d'allumages des leds si vous modifiez leur allumage à la fréquence de génération !
Afin d'avoir une séquence d'allumages aléatoires des leds qui soit perceptible visuellement, nous réaliserons l'appel à la fonction d'allumage des leds **périodiquement** grâce au *gestionnaire d'interruptions du périphérique systick*. Ceci pourra être réutilisé pour le TP 2.

Le *systick* est un timer, il peut déclencher une interruption périodiquement*, avec un intervalle de temps choisi.

/!\ Noter une particularité de ce contexte : les interruptions du systick ne sont pas gérées par le NVIC (Nested Vectored Interrupt Controller), et elles n'ont pas non plus besoin d'être annu-

* En réalité, lors du débordement d'un compteur interne. Voir par exemple http://embeddedsystemengineering.blogspot.com/2015/04/stm32f4-discovery-tutorial-4-make-delay.html

lées.

Grâce à cette solution, nous pourrons choisir une période beaucoup plus longue que celle de la génération des nombres, et chaque allumage utilisera simplement le dernier nombre aléatoire qui a été produit.
Vous **ne changerez rien** au code écrit jusqu'à la question 4, et vous **rajouterez** l'utilisation du timer. Autrement dit, nous aurons une exécution sommairement schématisée sur le dessin suivant :




Votre code écrit jusqu'à la question 4 récupère des valeurs produites par le RNG comme illustré en bleu. Sur les interruptions produites par le timer (en orange), l'allumage des leds est réalisé.

NOTE. Pour procéder à l'allumage des leds, le fichier *bp_leds.c* sera complété avec la définition de la fonction *write_leds_int* que vous avez réalisée à la semaine 3.

**Question 5.**
On notera que la fonction de la bibliothèque libopencm3 pour l'activation de l'interruption du systick est *systick_interrupt_enable()*, et que le gestionnaire d'interruptions associé porte le nom *sys_tick_handler*[**].

Il faut au préalable s'assurer que le périphérique *systick* a bien été configuré (par défaut, il n'est pas actif). Examiner le code de la fonction *systick_config* fournie et **compléter la ligne vide** selon les informations données ci-dessus.

L'objectif maintenant est de pouvoir provoquer des allumages des leds qui utilisent la valeur courante de la variable globale **`rnd`** dans laquelle chaque nouveau nombre est stocké.

Les valeurs possibles pour des allumages différents des 4 leds sont comprises entre 0 (tout éteint) et K (tout allumé). Quelle est cette valeur **K** ?

Ecrire **l'appel à la fonction *write_leds_int*** qui devra être utilisé pour que cette fonction reçoive en paramètre un nombre compris entre 0 et K-1, issu de la valeur de **`rnd`**.
(la valeur K pourrait ainsi être réservée pour signaler des cas d'erreur dans le RNG, dans la fonction de la question 3)

**Où** devrez-vous placer cet appel, et pourquoi ?

**Définir le gestionnaire d'interruptions** afin de réaliser l'action voulue, comme nous l'avons illustré en cours (cf slide 38).

Nous allons maintenant **tester** cette application terminée :

Cross-compiler et charger l'exécutable.
Placer un breakpoint sur le gestionnaire d'interruptions du *systick*, et un breakpoint sur l'instruction à l'entrée de la fonction *write_leds_int*.
Continuer l'exécution jusqu'à l'appel du gestionnaire d'interruptions du *systick*, et faire afficher la valeur de **`rnd`**. Puis continuer l'exécution jusqu'à l'entrée dans *write_leds_int* et faire afficher la valeur de **`intval`**. Vérifier la cohérence (selon le choix de calcul du paramètre fait ci-dessus) entre les valeurs de **`rnd`** et de **`intval`**.
Puis faire **`cont`** et vérifier que l'allumage des leds correspond bien à cette valeur.

Reproduire la même expérience quelques fois : observer les changements de valeurs de **`rnd`**, et aussi la cohérence entre l'allumage des leds et les valeurs de **`rnd`** et **`intval`**.

Finir en supprimant les 2 breakpoints, et observer l'allumage périodique aléatoire des leds.

---

[**] On notera aussi que les fonctions *systick_set_clocksource* et *systick_set_reload* sont utilisées pour fixer la fréquence à laquelle l'interruption sera déclenchée

## Annexe

```
/* This file is part of the libopencm3 project.
 *
 * This library is free software: you can redistribute it and/or modify
 * it under the terms of the GNU Lesser General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This library is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU Lesser General Public License for more details.
 *
 * You should have received a copy of the GNU Lesser General Public License
 * along with this library.  If not, see <http://www.gnu.org/licenses/>.
 */

/** @cond */
#ifdef LIBOPENCM3_RNG_H
/** @endcond */
#ifndef LIBOPENCM3_RNG_COMMON_F24_H
#define LIBOPENCM3_RNG_COMMON_F24_H

// #define MMIO32(addr)        (*(volatile uint32_t *)(addr))

/* --- Random number generator registers ---------------------------------- */

/* Control register */
#define RNG_CR                  MMIO32(RNG_BASE + 0x00)

/* Status register */
#define RNG_SR                  MMIO32(RNG_BASE + 0x04)

/* Data register */
#define RNG_DR                  MMIO32(RNG_BASE + 0x08)

/* --- RNG_CR values ------------------------------------------------------ */

/* RNG ENABLE */
#define RNG_CR_RNGEN            (1 << 2)

/* RNG interrupt enable */
#define RNG_CR_IE               (1 << 3)

/* --- RNG_SR values ------------------------------------------------------ */

/* Data ready */
#define RNG_SR_DRDY             (1 << 0)

/* Clock error current status */
#define RNG_SR_CECS             (1 << 1)

/* Seed error current status */
#define RNG_SR_SECS             (1 << 2)

/* Clock error interrupt status */
#define RNG_SR_CEIS             (1 << 5)

/* Seed error interrupt status */
#define RNG_SR_SEIS             (1 << 6)

#endif
/** @cond */
#else
#warning "rng_common_f24.h should not be included explicitly, only via rng.h"
#endif
/** @endcond */
```